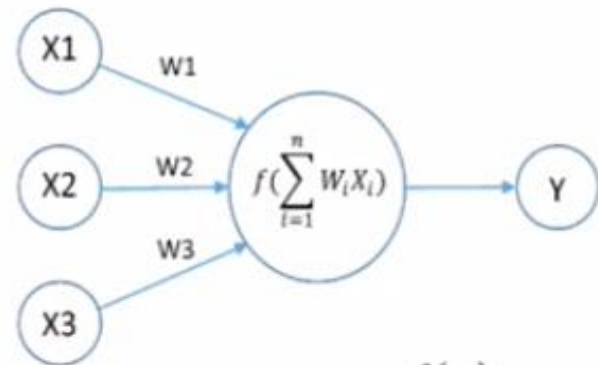
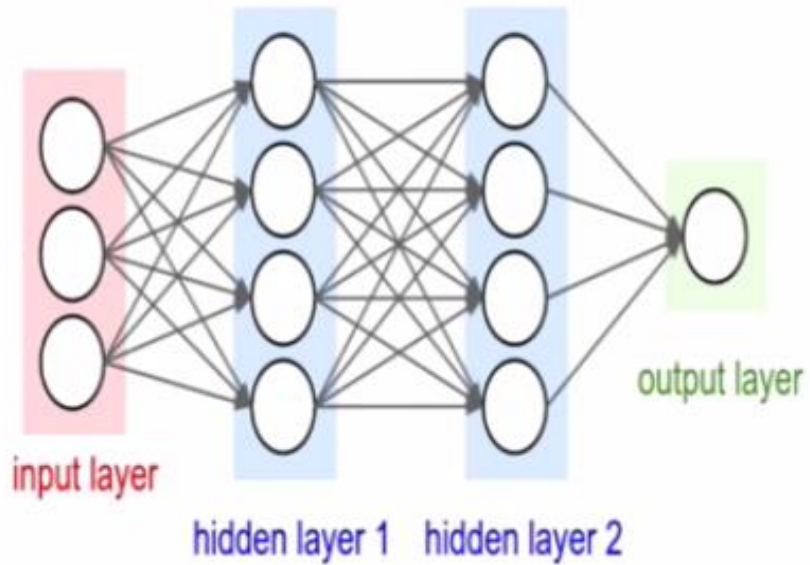


Intelligence Artificielle

Apprentissage automatique - réseau de neurones

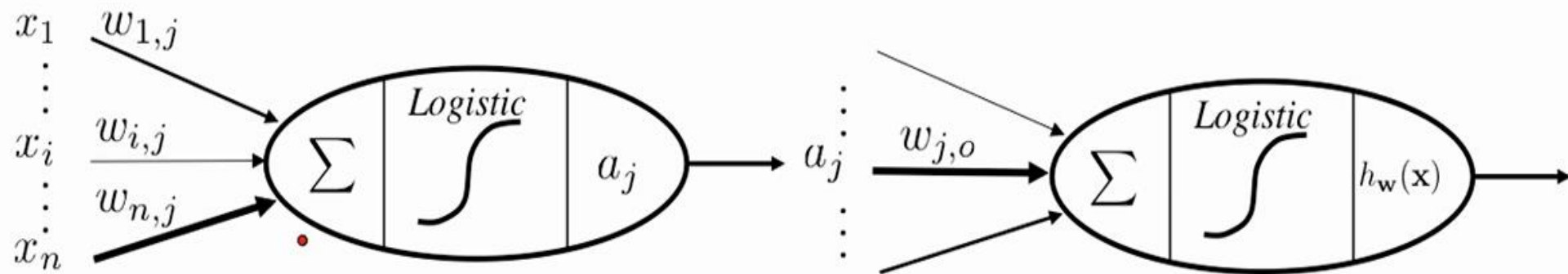
Réseau de neurones



$$f(x) = \frac{1}{1 + e^{-x}}$$

Quatrième algorithme: réseau de neurones artificiel

- **Idée:** apprendre les poids du classifieur linéaire **et** une transformation qui va rendre le problème linéairement séparable

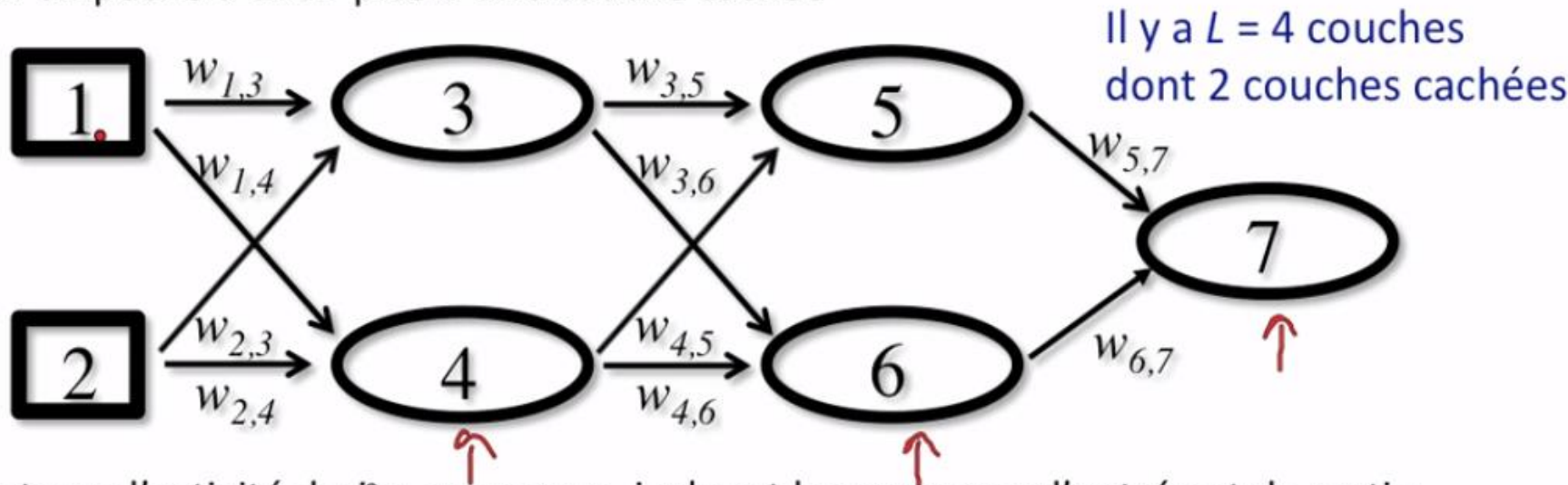


Réseau de neurones à une seule couche cachée



Cas général à L couches

- Rien n'empêche d'avoir plus d'une couche cachée

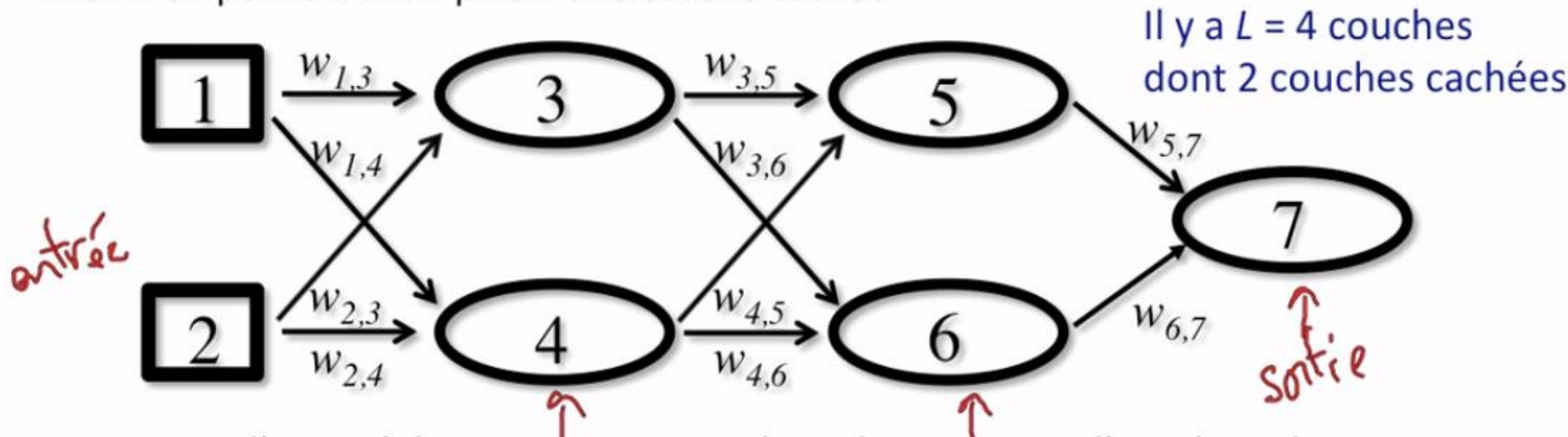


- On note a_j l'activité du j^{e} « neurone », incluant les neurones d'entrée et de sortie. Donc on aura $a_i = x_i$
- On note in_j l'activité du j^{e} neurone avant la non-linéarité logistique, c'est à dire

$$a_j = \text{Logistic}(in_j) = \text{Logistic}(\sum_i w_{i,j} a_i)$$

Cas général à L couches

- Rien n'empêche d'avoir plus d'une couche cachée

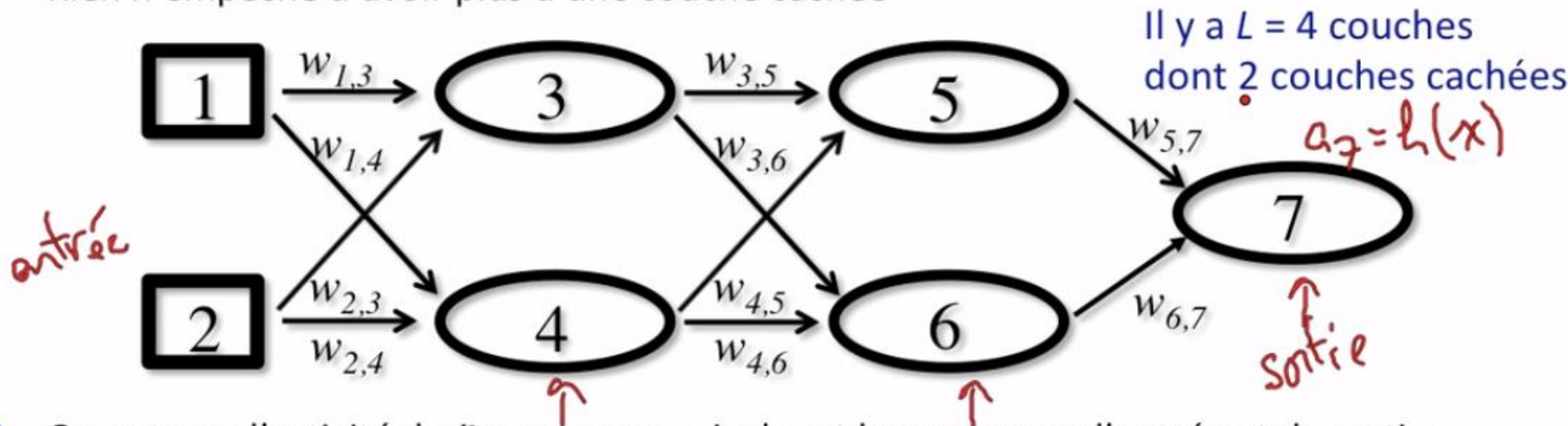


- On note a_j l'activité du j^{e} « neurone », incluant les neurones d'entrée et de sortie.
Donc on aura $a_i = x_i$
- On note in_j l'activité du j^{e} neurone avant la non-linéarité logistique, c'est à dire

$$a_j = Logistic(\boxed{in_j}) = Logistic(\sum_i \boxed{w_{ij}} \boxed{a_i})$$

Cas général à L couches

- Rien n'empêche d'avoir plus d'une couche cachée



- On note a_j l'activité du j^{e} « neurone », incluant les neurones d'entrée et de sortie. Donc on aura $a_i = x_i$
- On note in_j l'activité du j^{e} neurone avant la non-linéarité logistique, c'est à dire

$$a_j = \text{Logistic}(\boxed{in_j}) = \text{Logistic}(\sum_i \boxed{w_{ij}} \boxed{a_i})$$

← **rétropropagation**

$$\Delta[j] = g(in_j)(1 - g(in_j)) \sum w_{j,k} \Delta[k]$$